

Perfect. Începem cu un prototip TypeScript minimal.

Structura proiectului

```
typed-sfn/  
  package.json  
  tsconfig.json  
  src/  
    core.ts  
    examples.ts
```



1. Creează proiectul

</> Bash



```
mkdir typed-sfn  
cd typed-sfn  
npm init -y  
npm install typescript ts-node @types/node --save-dev  
npx tsc --init
```

2. `src/core.ts`

</> TypeScript



```
export type Idempotency = "idempotent" | "non-idempotent";  
  
export class Task<I, O> {  
  constructor(  
    public readonly name: string,  
    public readonly input: string,  
    public readonly output: string,  
    public readonly idempotency: Idempotency = "idempotent",  
    public readonly persistent: boolean = false,  
    public readonly compensation?: Task<any, any>  
  ) {}  
  
  nonIdempotent(): Task<I, O> {  
    return new Task(  
      this.name,  
      this.input,  
      this.output,  
      "non-idempotent",  
      this.persistent,  
      this.compensation  
    );  
  }  
  
  idempotentTask(): Task<I, O> {
```

```

        return new Task(
            this.name,
            this.input,
            this.output,
            "idempotent",
            this.persistent,
            this.compensation
        );
    }

    persistentTask(): Task<I, O> {
        return new Task(
            this.name,
            this.input,
            this.output,
            this.idempotency,
            true,
            this.compensation
        );
    }

    compensate<C extends Task<any, any>>(c: C): Task<I, O> {
        return new Task(
            this.name,
            this.input,
            this.output,
            this.idempotency,
            this.persistent,
            c
        );
    }
}

export function task<I, O>(
    name: string,
    input: string,
    output: string
): Task<I, O> {
    return new Task<I, O>(name, input, output);
}

export class Workflow<I, O> {
    constructor(public readonly tasks: Task<any, any>[]) {}
}

export function workflow<T extends Task<any, any>>[]>(
    ...tasks: T
): Workflow<any, any> {
    return new Workflow(tasks);
}

export type VerificationError =

```

```

| { kind: "ContractError"; message: string }
| { kind: "RetrySafetyError"; message: string }
| { kind: "CompensationError"; message: string };

export function verify(w: Workflow<any, any>): VerificationError[] {
  const errors: VerificationError[] = [];

  for (let i = 0; i < w.tasks.length - 1; i++) {
    const current = w.tasks[i];
    const next = w.tasks[i + 1];

    if (current.output !== next.input) {
      errors.push({
        kind: "ContractError",
        message: `${current.name} outputs ${current.output}, but ${next.name} expects ${n
      });
    }
  }

  for (const t of w.tasks) {
    if (t.idempotency === "non-idempotent" && (t as any).retry === true) {
      errors.push({
        kind: "RetrySafetyError",
        message: `${t.name} is non-idempotent and cannot be retried safely`,
      });
    }

    if (t.persistent && !t.compensation) {
      errors.push({
        kind: "CompensationError",
        message: `${t.name} is persistent but has no compensation handler`,
      });
    }
  }

  return errors;
}

export function retry<I, O>(t: Task<I, O>): Task<I, O> {
  return Object.assign(t, { retry: true });
}

export function compileToASL(w: Workflow<any, any>): object {
  const states: Record<string, any> = {};

  w.tasks.forEach((t, i) => {
    states[t.name] = {
      Type: "Task",
      Resource: `arn:aws:lambda:REGION:ACCOUNT:function:${t.name}`,
      ...(i < w.tasks.length - 1
        ? { Next: w.tasks[i + 1].name }
        : { End: true }),
    };
  });
}

```

```
    };  
  });  
  
  return {  
    StartAt: w.tasks[0].name,  
    States: states,  
  };  
}
```

3. src/examples.ts

❏ TypeScript



```
import { task, workflow, verify, retry, compileToASL } from "../core";  
  
type CustomerRequest = {};  
type OrderCreated = {};  
type OrderReserved = {};  
type OrderPaid = {};  
type OrderShipped = {};  
  
const createOrder = task<CustomerRequest, OrderCreated>(  
  "CreateOrder",  
  "CustomerRequest",  
  "OrderCreated"  
);  
  
const reserveInventory = task<OrderCreated, OrderReserved>(  
  "ReserveInventory",  
  "OrderCreated",  
  "OrderReserved"  
).persistentTask();  
  
const releaseInventory = task<OrderReserved, OrderCreated>(  
  "ReleaseInventory",  
  "OrderReserved",  
  "OrderCreated"  
);  
  
const chargeCard = task<OrderReserved, OrderPaid>(  
  "ChargeCard",  
  "OrderReserved",  
  "OrderPaid"  
).nonIdempotent();  
  
const shipOrder = task<OrderPaid, OrderShipped>(  
  "ShipOrder",  
  "OrderPaid",  
  "OrderShipped"  
);
```

```
const goodWorkflow = workflow(
  createOrder,
  reserveInventory.compensate(releaseInventory),
  chargeCard,
  shipOrder
);

console.log("Good workflow errors:");
console.log(verify(goodWorkflow));

console.log("Generated ASL:");
console.log(JSON.stringify(compileToASL(goodWorkflow), null, 2));

const badWorkflow = workflow(
  createOrder,
  shipOrder,
  retry(chargeCard)
);

console.log("Bad workflow errors:");
console.log(verify(badWorkflow));
```

4. Rulează

</> Bash



```
npx ts-node src/examples.ts
```

Ar trebui să vezi:

```
Good workflow errors:
[]

Bad workflow errors:
[
  ContractError...
  RetrySafetyError...
]
```



Următorul pas este să facem tipizarea mai puternică astfel încât unele erori să fie prinse chiar de TypeScript, nu doar de funcția `verify`.